News Summarization and Text-to-Speech Application

Name: Lahari Mullaguru
Email: laharimullaguru@gmail.com

Introduction

In an era of information overload, quickly extracting meaningful insights from news articles is critical. This application, developed as part of the Akaike Internship Assignment, aims to provide users with concise summarization, detailed sentiment analysis, comparative insights, and a user-friendly Hindi audio summary of news related to a company. The entire solution is available on GitHub and is deployed on Hugging Face Spaces.

Project Setup

Installation and Running Instructions

1. **Clone the Repository**

   git clone https://github.com/Lahari-Mullaguru/DataScience.git

   cd DataScience/NewsSummarization

2. **Install Dependencies**

   Dependencies are clearly mentioned in the requirements file:

   pip install -r requirements.txt

   Primary dependencies include:

   - Streamlit,
   - Uvicorn
   - BeautifulSoup4
   - NLTK
   - TextBlob
   - gTTS
   - deep-translator
   - python-dotenv.
   - FastAPI

3. **Setup Environment Variables**

   Create a .env file to securely store your NewsAPI key. I already have the .env file in my repository with my API key, you can test it further using any other key.

   NEWSAPI_API_KEY=your_api_key_here

4. **Run Backend (FastAPI)**

   python -m uvicorn api:app --reload

5. **Run Frontend (Streamlit App)**

   python -m streamlit run app.py

Model Details:

1. News Fetching – NewsAPI

Purpose: To retrieve the latest news articles related to a specific company.

How It Works:

- Sends a GET request to the NewsAPI endpoint (https://newsapi.org/v2/everything) with parameters like the search query (company_name), API key, language, and the number of articles to fetch.
- Parses the JSON response to extract article titles, summaries, and content for further processing.

Integration:

```
response = requests.get(API_ENDPOINT, params=params)
if response.status_code == 200:
    articles = response.json().get("articles", [])
    return [{
        "title": article["title"],
        "summary": article["description"],
        "content": article["content"]
    } for article in articles]
```

2. Sentiment Analysis - TextBlob

Purpose: To classify the sentiment of news articles as Positive, Negative, or Neutral.

How It Works:

- TextBlob leverages a pretrained lexicon-based sentiment analysis method.
- For each article, it calculates a polarity score: Polarity > 0 indicates **Positive sentiment** , Polarity < 0 **Negative** and Polarity = 0 indicates **Neutral sentiment**.

Integration:

```
blob = TextBlob(article["content"])
sentiment_score = blob.sentiment.polarity
if sentiment_score > 0:
    article["sentiment"] = "Positive"
elif sentiment_score < 0:
    article["sentiment"] = "Negative"
else:
    article["sentiment"] = "Neutral"
```

3. Topic Extraction - NLTK (Natural Language Toolkit)

Purpose: To dynamically extract the top three relevant topics (keywords) from each article.

How It Works:

- Tokenizes the text content into individual words.
- Removes common English stopwords (like 'the', 'is', 'at') to focus on meaningful words.
- Counts the frequency of remaining words and selects the top three most frequent words as key topics.

Integration:

```
stop_words = set(stopwords.words("english"))
words = word_tokenize(content.lower())
filtered_words = [word for word in words if word.isalnum() and word not in stop_words]

# Count word frequency and extract top 3 topics
word_freq = Counter(filtered_words)
topics = [word for word, _ in word_freq.most_common(3)]
return topics
```

4. Comparative Analysis

Purpose: To provide comparative insights across multiple news articles related to sentiment distribution and topic overlap.

How It Works:

- Computes the distribution of sentiment (Positive, Negative, Neutral) across all fetched articles.
- Identifies common topics appearing in at least two articles, and unique topics appearing only in individual articles.
- Dynamically generates comparative statements highlighting differences in coverage and sentiment.

5. Translation Model - GoogleTranslator (Deep Translator)

Purpose: Translates English text summaries to Hindi.

How It Works:

- Uses Google's translation API under the hood (through Deep Translator Python library) to translate English text into Hindi accurately.

Integration:

translated_text = GoogleTranslator(source="auto", target="hi").translate(summary_text)

6. Text-to-Speech (TTS) - gTTS

Purpose: Converts textual summaries into spoken audio files in Hindi.

How It Works:

- Converts translated Hindi text into speech using Google's TTS services.
- Generates an MP3 audio file, saved locally or returned as a base64 encoded string for deployment purposes in hugging face.

Integration:

- tts = gTTS(translated_text, lang="hi", slow=False)
- tts.save(mp3_filename)

Library	Purpose	Strengths	Limitations
NewsAPI	Fetch relevant news articles	Fast, accurate, simple integration	Requires API Key, usage limits
TextBlob	Sentiment analysis	Quick, easy implementation	Less accurate on nuanced texts
NLTK	Tokenization and topic extraction	Robust NLP toolkit, accurate tokenizing	Frequency-based extraction
GoogleTranslator	Translate summaries to Hindi	High accuracy, easy usage	Online-only, limited with complex texts
gTTS	Generate Hindi audio from text summaries	Clear audio, multi-language support	Dependent on Google services, online-only

API Development:

The application's backend is powered by **FastAPI**, exposing a clearly defined REST API endpoint at:

curl -X GET "http://127.0.0.1:8000/analyze-news?company_name=Tesla"

It processes the company name provided and returns structured JSON data containing article summaries, sentiment analysis, comparative insights, and the TTS audio URL.

Testing API via Postman

- Method: **GET**
- Endpoint: http://127.0.0.1:8000/analyze-news?company_name=Tesla
- Parameter: company_name (e.g., Tesla, Apple, Amazon)
- Response: JSON data with analysis details.

API Usage (Third-party API):

API Name	Purpose	Integration Method	Required Libraries
NewsAPI	Fetch news articles	RESTful API (HTTP GET request)	requests, python-dotenv
Google TTS (gTTS)	Text-to-Speech conversion (Hindi audio)	Python library integration (gtts)	gtts, deep-translator

Hugging Face Deployment:

For deploying the application to Hugging Face Spaces, minor adjustments were made:

- Utilized threading within Streamlit's frontend (app.py) to initiate the FastAPI backend efficiently. This threading mechanism ensures that both backend and frontend can run simultaneously without separate manual intervention.
- Modified the utility functions (utils.py) to handle audio files as base64 encoded strings, enabling smoother integration with Hugging Face Spaces without static file issues.

Assumptions & Limitations

Assumptions:

- NewsAPI and Google's TTS (via gTTS) services are consistently available.
- TextBlob accurately identifies article sentiment.
- Google Translator accurately translates summaries to Hindi.

Limitations:

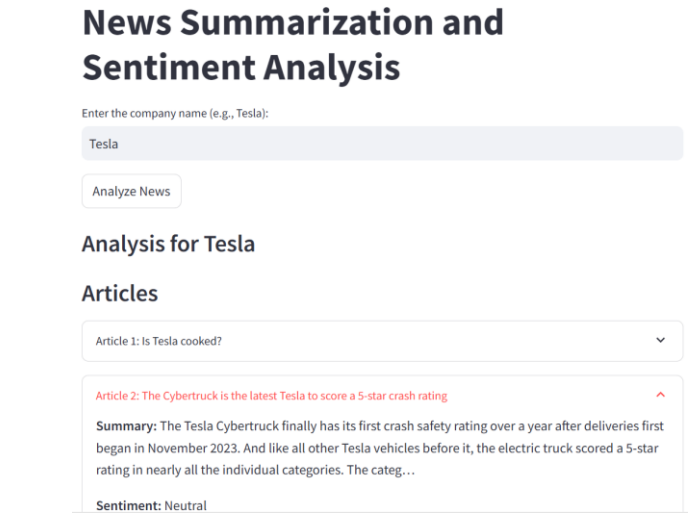
- Sentiment analysis may overlook context and nuances.
- Topic extraction relies on basic frequency counting, potentially missing meaningful context.

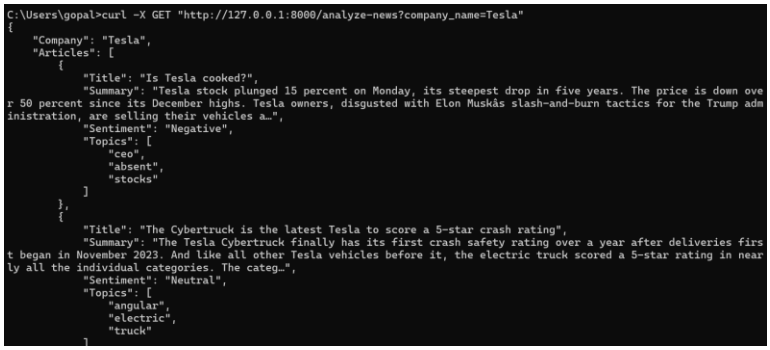
- The TTS functionality (gTTS) requires an internet connection and may face service limitations or downtime.

Challenges Encountered

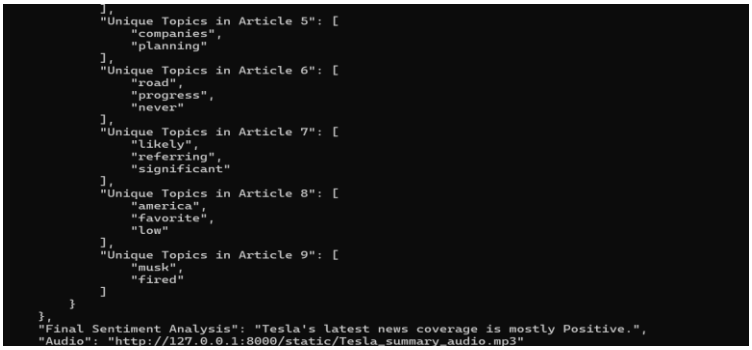
While striving to strictly adhere to open-source tools for Text-to-Speech (TTS) as per the original assignment requirement, significant challenges arose. During the setup of the Coqui TTS library (pip install TTS), I encountered persistent wheel dependency errors despite repeated installations of all recommended libraries and dependencies. After thorough investigation and multiple troubleshooting attempts, I opted for a reliable alternative gTTS.

Output:





```
C:\Users\gopal>curl -X GET "http://127.0.0.1:8000/analyze-news?company_name=Tesla"
{
    "Company": "Tesla",
    "Articles": [
        {
            "Title": "Is Tesla cooked?",
            "Summary": "Tesla stock plunged 15 percent on Monday, its steepest drop in five years. The price is down ove
r 50 percent since its December highs. Tesla owners, disgusted with Elon Muskâs slash-and-burn tactics for the Trump adm
inistration, are selling their vehicles a…",
            "Sentiment": "Negative",
            "Topics": [
                "ceo",
                "absent",
                "stocks"
            ]
        },
        {
            "Title": "The Cybertruck is the latest Tesla to score a 5-star crash rating",
            "Summary": "The Tesla Cybertruck finally has its first crash safety rating over a year after deliveries firs
t began in November 2023. And like all other Tesla vehicles before it, the electric truck scored a 5-star rating in near
ly all the individual categories. The categ…",
            "Sentiment": "Neutral",
            "Topics": [
                "angular",
                "electric",
                "truck"
            ]
        }
    ],
    "Comparisons": [
        {
            "Comparison": "Article 6 discusses road, progress, never with a positive sentiment, while Article 7 disc
usses likely, referring, significant with a positive sentiment.",
            "Impact": "Both articles provide a balanced perspective on the company's current situation."
        },
        {
            "Comparison": "Article 7 discusses likely, referring, significant with a positive sentiment, while Artic
le 8 discusses america, favorite, low with a negative sentiment.",
            "Impact": "The first article highlights positive developments, boosting confidence in the company's grow
th, while the second article raises concerns about potential challenges."
        },
        {
            "Comparison": "Article 8 discusses america, favorite, low with a negative sentiment, while Article 9 dis
cusses musk, tesla, fired with a positive sentiment.",
            "Impact": "The first article raises concerns about challenges, while the second article highlights posit
ive developments, boosting confidence in the company's growth."
        }
    ],
    "Topic Overlap": {
        "Common Topics": [
            "tesla"
        ],
        "Unique Topics in Article 1": [
            "ceo",
            "absent",
            "stocks"
        ],
```



Conclusion

The developed application effectively integrates modern NLP and API technologies to deliver valuable insights in a user-friendly manner. Despite certain technical limitations and dependency hurdles, the final solution meets the project's core requirements of summarizing news, analyzing sentiments, providing comparative insights, and generating user-friendly audio summaries. The integration with Hugging Face Spaces demonstrates robustness and practical applicability, ensuring accessibility and ease of use.

Links:

- **GitHub Repository**: NewsSummarization
- **Deployment on Hugging Face Spaces**: News Summarization App